

AAFPN — Comprendre les corrections

Guide pédagogique • React, TypeScript, Tailwind et Git

Du premier problème visible dans cette conversation à la publication sur main. Version du 23 septembre 2026, fondée sur le code local et le commit c345c80.

Ce que ce document couvre

Les problèmes de présentation de EventsSection, leurs corrections, la disparition des boutons de pagination, puis le renommage de branche et la publication Git. Le composant EventsSection est expliqué bloc par bloc et reproduit intégralement en annexe avec ses numéros de ligne.

Une limite importante

Le premier problème documenté ici est la demande de vérification et de réorganisation de EventsSection. Les anciens commits du backend existent, mais leurs échanges de débogage ne figurent pas dans cette conversation : ce guide ne leur invente pas de bugs ni de solutions. Hero, SidebarNav et les services étaient déjà présents lorsque cette intervention a commencé ; leur rôle est expliqué sans attribuer leur création à cette intervention.

Parcours de lecture

1. Les problèmes et solutions
2. Le circuit des données
3. Les états React
4. Chargement, filtre et nettoyage
5. Pagination et boutons
6. Structure et accessibilité
7. Cartes et images
8. Styles et dates
9. Autres fichiers
10. Git et vérifications
11. Exercices et pistes restantes
12. Code intégral commenté par renvoi aux chapitres

1. Les problèmes et leurs solutions

Cartes trop serrées sur mobile

Avant : les cartes restaient en ligne, avec une image occupant 42 % de la largeur. Sur un téléphone, le texte recevait peu de place. Solution : flex-col empile l'image et les informations ; sm:flex-row rétablit la disposition horizontale sur un écran plus large. min-w-0 et wrap-anywhere permettent au texte long de se réduire et de revenir à la ligne.

Navigation incohérente

Avant : deux flèches verticales placées à gauche, dont une sur fond jaune avec un padding différent. Solution : deux boutons ronds bleus de mêmes dimensions sous les cartes, avec flèches gauche/droite et compteur. Les états désactivé, survol et focus clavier sont explicités.

Image de remplacement introuvable

Avant : /images/events-placeholder.jpg était utilisé, mais ce fichier était absent. Solution : le logo existant sert de remplacement. Une erreur de chargement de l'image distante déclenche également ce remplacement.

Dernière page répétant un événement

Avant : la limite était `events.length - ITEMS_PER_PAGE`. Avec cinq événements et deux par page, les indices de début devenaient 0, 2, 3 : le quatrième événement apparaissait deux fois. La limite est désormais le début de la dernière page : 0, 2, 4. Le dernier écran ne contient alors que le cinquième événement.

Chargement et démontage

Un indicateur active a été ajouté pour ignorer le résultat d'une requête devenue obsolète après le nettoyage de l'effet. Le titre et le conteneur sont conservés pour les états de chargement, d'erreur et de liste vide.

2. Comment les données arrivent

```
App → HomePage → EventsSection
    ↓ getEvents()
    instance Axios
    ↓ GET /events
    API backend
    ↓ response.data.events
    filtre → tri → slice → cartes
```

La configuration

frontend/src/config.ts lit VITE_API_BASE_URL. Si elle est absente ou vide, la valeur utilisée est `http://localhost:3000/api`. `replace(/V$/, '')` enlève un slash final pour faciliter l'assemblage des chemins. `API_ORIGIN` enlève le suffixe `/api`, mais `EventsSection` ne s'en sert pas.

Le service HTTP

`api.ts` crée une instance `Axios` avec cette `baseURL`. `event.services.ts` appelle `api.get("/events")` : avec la configuration par défaut, la requête vise `http://localhost:3000/api/events`. `await` attend la réponse ; la fonction renvoie une `Promise<Event[]>`.

La validation actuellement effectuée

`Array.isArray(response.data.events)` vérifie que `events` est un tableau. Sinon, le service déclenche une erreur reprise dans le `catch` du composant. Le type générique `Axios` décrit la réponse pour TypeScript ; il ne valide pas chaque champ à l'exécution. Une date ou un statut mal formé peut donc encore arriver dans le composant.

Séparer les responsabilités

Le service s'occupe de récupérer les données ; `EventsSection` décide lesquelles montrer et comment les présenter ; `EventCard` présente un événement. Ce découpage évite de mélanger l'adresse du serveur, la pagination et le contenu d'une carte.

3. Imports, composant et états

```
import { useEffect, useState } from "react";
import { getEvents } from "../../services/event.services.ts";
import type { Event } from "../../types/event.ts";
const ITEMS_PER_PAGE = 2;
```

useState conserve une valeur entre les rendus et déclenche un nouveau rendu quand elle change. useEffect relie le composant à une opération extérieure, ici une requête HTTP. import type ne fournit qu'un type TypeScript : il ne crée pas de valeur JavaScript. ITEMS_PER_PAGE est une constante partagée par l'affichage et les calculs.

```
const [events, setEvents] = useState<Event[]>([]);
const [startIndex, setStartIndex] = useState(0);
const [loading, setLoading] = useState(true);
const [error, setError] = useState<string | null>(null);
```

events contient les événements retenus, et non toutes les données brutes. Event[] signifie « tableau d'événements ». startIndex indique le premier élément de la page courante ; les indices commencent à zéro. loading vaut true avant la fin du premier chargement. error vaut null lorsqu'aucune erreur ne doit être affichée, sinon une chaîne lisible.

Pourquoi deux valeurs entre crochets ?

La déstructuration sépare la valeur et sa fonction de mise à jour. setEvents(liste) demande à React de refaire le rendu avec cette liste. On ne modifie pas directement events avec une affectation. Les calculs visibleEvents et hasImage restent des valeurs dérivées : ils sont recalculés, sans état supplémentaire inutile.

export default function EventsSection()

Cette fonction est le composant exporté par défaut du fichier. HomePage peut l'importer et l'utiliser avec <EventsSection />. Son return décrit l'interface en JSX ; les expressions entre accolades sont évaluées en JavaScript.

4. Chargement et cycle de vie

```
useEffect(() => {  
  let active = true;  
  async function loadEvents() { /* try / catch / finally */ }  
  void loadEvents();  
  return () => { active = false; };  
}, []);
```

Le tableau de dépendances vide signifie que cet effet ne dépend d'aucune valeur réactive du composant. Il s'installe au montage. En développement, StrictMode peut provoquer un cycle supplémentaire installation/nettoyage pour détecter les effets fragiles. Il ne faut donc pas supposer qu'une seule requête réseau sera toujours envoyée.

try : le chemin normal

setLoading(true) et setError(null) préparent l'état de chargement. await getEvents() récupère la liste. new Date() capture l'heure de référence du filtrage. filter garde uniquement les événements publiés dont la fin — ou, faute de fin, le début — n'est pas passée. sort les classe par début croissant.

```
event.status === "PUBLISHED" &&  
new Date(event.endAt ?? event.startAt) >= now
```

=== compare exactement le statut ; && exige les deux conditions. ?? prend startAt uniquement lorsque endAt est null ou undefined. Un événement sans endAt disparaît dès que son début est passé. Un événement en cours avec une fin future reste affiché. Le filtrage n'est pas réévalué automatiquement à mesure que l'heure avance.

catch, finally et nettoyage

catch intercepte les erreurs du chargement et de la préparation des données, les journalise, puis affiche un message simple. finally s'exécute après succès ou échec et termine le chargement. Les mises à jour finales sont protégées par if (active). Le nettoyage met active à false : les résultats tardifs ne mettent plus à jour cet effet. Il n'annule pas la requête HTTP.

void loadEvents() lance la fonction sans attendre sa Promise dans le callback de useEffect. Le mot void n'intercepte pas une erreur ; ici le try/catch interne assure la gestion.

5. Pagination : comprendre les calculs

```
const visibleEvents = events.slice(
  startIndex, startIndex + ITEMS_PER_PAGE
);
```

slice extrait une portion sans modifier events. Le premier indice est inclus, le second exclu. Avec startIndex = 2 et une taille de 2, les indices 2 et 3 sont visibles, donc les troisième et quatrième événements.

Reculer sans indice négatif

```
setStartIndex(previous => Math.max(previous - 2, 0));
```

La fonction de mise à jour reçoit la dernière valeur de l'état. Math.max impose le minimum 0. Depuis 2 on revient à 0 ; depuis 0 on reste à 0. Dans le fichier, le nombre 2 est remplacé par ITEMS_PER_PAGE.

Avancer sans chevaucher les pages

```
lastStart = Math.max((Math.ceil(total / size) - 1) * size, 0)
nextStart = Math.min(previous + size, lastStart)
```

Ces deux lignes sont une reformulation pédagogique de la formule du code. Math.ceil arrondit le nombre de pages vers le haut ; on retire 1 pour obtenir l'indice de la dernière page, puis on multiplie par la taille. Pour cinq événements : $\text{ceil}(5/2) = 3$ pages, donc dernier début = $(3 - 1) \times 2 = 4$.

Total	Indices de début	Éléments affichés
2	0	1–2
3	0, 2	1–2 puis 3
5	0, 2, 4	1–2 puis 3–4 puis 5

Pourquoi les boutons étaient invisibles

```
events.length > ITEMS_PER_PAGE && (...)
```

Les boutons existent uniquement si la liste filtrée contient plus de deux événements. Deux cartes visibles ne nécessitent pas une deuxième page. Les brouillons, annulations et événements terminés ne comptent pas. La proposition ITEMS_PER_PAGE = 1 a été expliquée, mais pas appliquée : le fichier actuel conserve 2.

Précédent est désactivé à startIndex === 0. Suivant est désactivé lorsque startIndex + ITEMS_PER_PAGE >= events.length. Le compteur utilise startIndex + 1 pour passer d'un indice technique à une position compréhensible.

6. Le rendu et l'accessibilité

Un cadre commun à tous les états

section porte `id="events"` : le lien `#events` du menu peut l'atteindre. `aria-labelledby="events-heading"` donne au bloc le titre du `h2` comme nom accessible. `aria-busy={loading}` indique qu'il est en cours de mise à jour. Le conteneur centré limite la largeur des cartes.

```
loading ? chargement
: error ? erreur
: events.length === 0 ? listeVide
: contenu
```

Les ternaires imbriqués sélectionnent un seul état. Le chargement est prioritaire, puis l'erreur, puis la liste vide. `<>...</>` est un fragment React : il regroupe plusieurs éléments sans ajouter de `div` dans le DOM.

Le filigrane et la liste

Le logo décoratif est en position absolue dans un parent relative. Son opacité de 5 % le rend discret. `pointer-events-none` empêche de bloquer les clics. `alt=""` et `aria-hidden="true"` le retirent de la lecture utile. `visibleEvents.map` crée une `EventCard` par événement ; `key={event.id}` permet à React de suivre l'identité des cartes.

Les commandes

`nav` possède un nom accessible. `type="button"` empêche une soumission de formulaire involontaire. `onClick` reçoit la fonction à exécuter au clic ; on n'écrit pas `previousEvents()` dans cette prop, car cela l'appellerait pendant le rendu. `disabled` bloque réellement l'interaction aux limites.

`aria-label` nomme les flèches, tandis que `aria-hidden` masque leur symbole décoratif aux lecteurs d'écran. `aria-controls="events-list"` relie chaque commande à la liste concernée. `focus-visible` ajoute une bordure de focus visible lors de la navigation au clavier.

Les annonces

`role="status"` convient au chargement et au compteur pour une annonce non urgente. `aria-atomic="true"` demande de lire le compteur entier lorsqu'il change. `role="alert"` sert au message d'erreur. Ces attributs améliorent l'information accessible ; ils ne remplacent pas un test réel au clavier et au lecteur d'écran.

7. EventCard et les images

```
type EventCardProps = { event: Event };
function EventCard({ event }: EventCardProps) { ... }
```

Les props sont les données transmises par le parent. { event } extrait l'événement de l'objet reçu. EventCard ne récupère pas de données réseau : elle reçoit un événement déjà choisi par EventsSection.

```
const [failedImageUrl, setFailedImageUrl] =
  useState<string | null>(null);
const hasImage = Boolean(
  event.imageUrl && event.imageUrl !== failedImageUrl
);
```

failedImageUrl mémorise l'adresse ayant échoué. hasImage vaut true seulement si une adresse est fournie et n'a pas déjà échoué. Boolean convertit le résultat de l'expression en vrai ou faux. Mémoriser l'adresse plutôt qu'un simple booléen autorise une nouvelle URL à être essayée.

src, onError et le logo

src choisit l'image de l'événement si hasImage est vrai, sinon /images/logo-couleur.png. L'opérateur ! après imageUrl est une assertion TypeScript de non-nullité ; il ne vérifie rien à l'exécution. Ici, la branche hasImage repose sur la présence de l'adresse.

onError mémorise l'URL en échec. Un nouveau rendu affiche alors le logo. Le gestionnaire devient undefined pour ce logo : son éventuel échec ne déclenche pas une boucle de remplacement. loading="lazy" demande au navigateur de différer le chargement des images éloignées de la zone visible.

Texte alternatif et limites

alt="" traite l'image comme illustrative, le titre étant déjà affiché à côté. Si une affiche contient des informations importantes absentes du texte, il faudra prévoir une description appropriée dans les données. Les chemins relatifs d'images reçus du backend ne sont pas automatiquement préfixés avec API_ORIGIN dans ce composant.

Titre, description et détails

h3 affiche event.title. La description n'est rendue que si elle est non vide. dl contient des paires dt/dd : nom de l'information et valeur. time fournit startAt comme valeur lisible par les machines. calendarDateLabel est préféré au formatage standard lorsqu'il est non vide. location?.trim() enlève les espaces si le lieu existe ; || affiche « À préciser » en cas de résultat vide.

8. Les styles et les dates

Lire les classes Tailwind

Classes	Rôle dans le composant
relative / absolute / inset-0	Positionner le filigrane dans son parent.
mx-auto / max-w-4xl	Centrer et limiter la largeur du contenu.
px-5 / py-16 / p-5	Espacement intérieur horizontal, vertical ou global.
flex-col / sm:flex-row	Empiler sur petit écran ; aligner dès le palier sm.
sm:w-[38%] / shrink-0	Réserver la largeur de l'image sans la comprimer.
min-w-0 / flex-1	Autoriser le texte à rétrécir et occuper l'espace restant.
object-cover / object-contain	Remplir en recadrant / montrer l'image entière.
whitespace-pre-line	Conserver les retours à la ligne de la description.
wrap-anywhere	Couper si nécessaire un mot ou une URL trop longue.
grid-cols-[auto_minmax(0,1fr)]	Colonne des libellés puis colonne flexible des valeurs.
rounded-xl / overflow-hidden	Arrondir la carte et contenir son image.
bg-white/25 / hover:shadow-lg	Fond blanc à 25 % et ombre au survol dans le code actuel.

Les préfixes sm: et md: sont des conditions de largeur minimale. Sans personnalisation des paliers, ils correspondent respectivement à 40rem et 48rem. Le style de base s'applique d'abord, puis les variantes sur les écrans plus larges.

Trois fonctions de formatage

`formatDate` construit un `Intl.DateTimeFormat` en français avec jour, mois en lettres et année. `formatTime` choisit heure et minute sur deux chiffres. Les deux imposent Europe/Paris pour que l'affichage ne dépende pas du fuseau du navigateur. `formatTimeRange` affiche seulement le début sans `endAt`, sinon « début – fin ».

Une date invalide peut faire échouer `Intl.DateTimeFormat` pendant le rendu : ce cas n'est pas couvert par le catch du chargement. Une plage traversant plusieurs jours n'affiche actuellement que les heures, sans la date de fin. Ce sont des améliorations possibles, non réalisées ici.

9. Les autres fichiers du frontend

App.tsx et HomePage.tsx

App affiche HomePage. HomePage assemble Hero puis EventsSection dans un fragment. La fonction de HomePage s'appelle actuellement App en interne : son export par défaut permet de l'importer sous le nom HomePage, mais renommer la fonction améliorerait la lisibilité. Ce renommage n'a pas été effectué.

Hero.tsx

Le hero utilise une section relative de hauteur minimale égale à l'écran, une photo en absolute avec object-cover, un filtre bleu, puis un contenu au-dessus avec z-10. Le logo et le texte visuel sont centrés. SidebarNav est inséré dans cette section, mais son positionnement fixed l'attache à la fenêtre.

SidebarNav.tsx

menuOpen est un booléen React. Quand il est faux, le bouton d'ouverture est visible ; quand il est vrai, le nav avec les liens et le bouton de fermeture est rendu. navigation.map crée les liens, key utilise leur href, et closeMenu referme le menu au clic. Les liens #about, #courses et #contact nécessitent des sections correspondantes ; HomePage ne compose actuellement que Hero et EventsSection.

Types et CSS : observations non corrigées

EventStatus contient « PUBLISHED », alors que le filtre attend « PUBLISHED ». Pour le moment, Event.status est déclaré string, donc cette faute dans l'union ne bloque pas le filtre à la compilation. Une correction cohérente serait de corriger l'union puis de typer status avec EventStatus, après vérification du contrat backend.

index.css contient merge: 0 dans body, vraisemblablement à la place de margin: 0. merge n'est pas une propriété CSS valide. Cette anomalie n'a pas été corrigée dans la séquence décrite. Les outils peuvent réussir malgré certaines déclarations CSS sans effet.

Ce qui a été enregistré

Le commit contient 22 fichiers modifiés ou ajoutés : composants, services, types, configuration, images et dépendances. Tous ces fichiers ont été inclus à la demande de publication de l'ensemble des développements frontend ; ils n'ont pas tous été écrits pendant la correction de EventsSection.

10. Git : de la branche à main

Situation de départ

La branche active était develop/vitrin-page. Elle et main pointaient initialement vers 27911c9. Le dossier de travail contenait des fichiers modifiés et de nouveaux fichiers frontend non suivis. Le dépôt distant origin correspond à github.com/Beehnood/AAFPN.

```
git fetch origin
git branch -m develop/hero-nav-events
git add frontend
git commit -m "feat(frontend): ajouter le hero, la navigation et les événements"
git push -u origin develop/hero-nav-events
```

fetch met à jour les références distantes sans fusionner. branch -m renomme la branche locale courante. add prépare les fichiers ; commit enregistre un instantané, ici c345c80. push publie les commits ; -u relie la branche locale à sa branche distante. Le renommage local ne supprime pas automatiquement l'ancienne branche distante develop/vitrin-page, qui n'a pas été supprimée.

```
git switch main
git pull --ff-only origin main
git merge --ff-only develop/hero-nav-events
git push origin main
git pull --ff-only origin main
git status --short --branch
```

switch change la branche active. Le premier pull actualise main avant intégration. merge --ff-only avance main vers le commit de développement si l'historique le permet, sans créer de commit de fusion. Il échoue en cas de divergence. Le push publie main ; le dernier pull confirme sa synchronisation. Aucun push forcé n'a été utilisé.

Vérifications réellement exécutées

npm run build a exécuté tsc -b puis vite build avec succès. npm run lint a exécuté ESLint avec succès. Le contrôle git diff --check avait signalé des espaces en fin de ligne dans App.tsx et une ligne vide finale dans index.css ; ils n'ont pas bloqué ces commandes. Aucun test automatisé de navigation ni contrôle visuel complet n'a été réalisé.

À la fin de la publication, main était synchronisée avec origin/main et le dossier de travail était propre. Le frontend sur le port 5173 et le backend sur le port 3000 fonctionnaient déjà ; le navigateur a été ouvert sur <http://localhost:5173>. Un push Git n'est pas, à lui seul, un déploiement du site en production.

11. S'entraîner et diagnostiquer

Exercice 1 — Une seule carte par page

Dans une branche de travail, remplacer `ITEMS_PER_PAGE = 2` par 1. Prévision : avec trois événements admissibles, les indices sont 0, 1, 2 et le compteur progresse de 1–1 à 3–3. Les boutons restent absents s'il n'y a qu'un événement. Revenir à 2 après l'essai si c'est le choix de présentation souhaité.

Exercice 2 — Un nombre impair

Avec cinq événements publiés et non terminés, vérifier les pages 1–2, 3–4, puis 5. Revenir deux fois en arrière. Chaque événement doit apparaître une fois dans le parcours avant ; aucun bouton ne doit dépasser les limites.

Exercice 3 — Image en échec

Utiliser sur des données de test une adresse d'image inexistante. Le logo doit apparaître après l'échec réseau ; le texte reste lisible. Tester aussi `imageUrl` absent, null et vide. Vérifier ensuite un titre très long et une description sur plusieurs lignes.

Exercice 4 — Comprendre un écran vide

Distinguer le message de chargement, le message d'erreur et « Aucun événement à venir ». Dans les outils réseau du navigateur, contrôler la requête `/events`, son statut HTTP et la forme de sa réponse. Puis vérifier `PUBLISHED`, `endAt` et `startAt`. Une API qui renvoie des données ne garantit pas qu'elles passent le filtre.

Exercice 5 — Vérifier l'interface

Tester une largeur de 320 px, une tablette et un grand écran. Les cartes doivent passer de verticales à horizontales. Utiliser Tab puis Entrée sur les boutons ; vérifier le focus visible. Examiner la dernière page et le compteur. Ces essais sont proposés pour apprendre : ils ne sont pas présentés comme déjà effectués.

Améliorations envisageables

Valider chaque donnée API, gérer les dates invalides et les événements sur plusieurs jours, annuler réellement les requêtes si nécessaire, normaliser les URL d'images, ajouter un bouton Réessayer et vérifier le contraste du menu fixe sur fond clair. Ces pistes ne font pas partie des corrections livrées.

Lire le code intégral

L'annexe suivante reproduit le fichier local `EventsSection.tsx` au moment de la création de ce guide. Les numéros de ligne sont des repères de lecture, pas du code à copier. Les longues lignes sont repliées uniquement pour l'impression.

Annexe — EventsSection.tsx (1)

Source intégrale • explications dans les chapitres 3 à 8

```
1 import { useEffect, useState } from "react";
2
3 import { getEvents } from "../../services/event.services.ts";
4 import type { Event } from "../../types/event.ts";
5
6 const ITEMS_PER_PAGE = 2;
7
8 export default function EventsSection() {
9   const [events, setEvents] = useState<Event[]>([]);
10  const [startIndex, setStartIndex] = useState(0);
11
12  const [loading, setLoading] = useState(true);
13  const [error, setError] = useState<string | null>(null);
14
15  useEffect(() => {
16    let active = true;
17
18    async function loadEvents() {
19      try {
20        setLoading(true);
21        setError(null);
22
23        const data = await getEvents();
24
25        const now = new Date();
26
27        const upcomingEvents = data
28          .filter((event) => {
29            return (
30              event.status === "PUBLISHED" &&
31              new Date(event.endAt ?? event.startAt) >= now
32            );
33          })
34          .sort(
35            (a, b) =>
36              new Date(a.startAt).getTime() -
37              new Date(b.startAt).getTime()
38          );
39
40        if (active) setEvents(upcomingEvents);
41      } catch (error) {
42        console.error("Erreur événements :", error);
43
44        if (active) {
45          setError("Impossible de charger les événements pour le moment.");
46        }
47      } finally {
48        if (active) setLoading(false);
```

Annexe — EventsSection.tsx (2)

Source intégrale • explications dans les chapitres 3 à 8

```
49     }
50   }
51
52   void loadEvents();
53
54   return () => {
55     active = false;
56   };
57 }, []);
58
59 const visibleEvents = events.slice(
60   startIndex,
61   startIndex + ITEMS_PER_PAGE
62 );
63
64 function previousEvents() {
65   setStartIndex((previous) =>
66     Math.max(previous - ITEMS_PER_PAGE, 0)
67   );
68 }
69
70 function nextEvents() {
71   setStartIndex((previous) =>
72     Math.min(
73       previous + ITEMS_PER_PAGE,
74       Math.max(Math.ceil(events.length / ITEMS_PER_PAGE) - 1) * ITEMS_PER_PAGE, 0)
75   )
76 };
77 }
78
79 return (
80   <section
81     id="events"
82     aria-labelledby="events-heading"
83     aria-busy={loading}
84     className="relative overflow-hidden bg-white px-5 py-16 sm:px-8 md:py-24"
85   >
86     <div className="mx-auto max-w-4xl">
87       <h2
88         id="events-heading"
89         className="mb-10 text-center text-2xl font-bold uppercase tracking-wide
90 text-[#0077c8] md:mb-12 md:text-3xl"
91       >
92         Nos événements
93       </h2>
94
95       {loading ? (
96         <p role="status" className="flex min-h-64 items-center justify-center text-center
```

Annexe — EventsSection.tsx (3)

Source intégrale • explications dans les chapitres 3 à 8

```

    text-[#0077c8]">
96      Chargement des événements...
97    </p>
98    ) : error ? (
99      <p role="alert" className="flex min-h-64 items-center justify-center text-center
text-red-600">
100        {error}
101      </p>
102    ) : events.length === 0 ? (
103      <p className="flex min-h-64 items-center justify-center text-center text-
gray-500">
104        Aucun événement à venir pour le moment.
105      </p>
106    ) : (
107      <>
108        <div className="relative">
109          
115          <div id="events-list" className="relative grid gap-6">
116            {visibleEvents.map((event) => (
117              <EventCard key={event.id} event={event} />
118            ))}
119          </div>
120        </div>
121
122        {events.length > ITEMS_PER_PAGE && (
123          <nav aria-label="Pagination des événements" className="mt-8 flex items-center
justify-center gap-4 sm:gap-6">
124            <button
125              type="button"
126              onClick={previousEvents}
127              disabled={startIndex === 0}
128              aria-label="Événements précédents"
129              aria-controls="events-list"
130              className="flex size-11 shrink-0 items-center justify-center rounded-full
border border-[#0077c8]/30 text-[#0077c8] transition-colors enabled:hover:bg-[#0077c8]
enabled:hover:text-white focus-visible:outline-2 focus-visible:outline-offset-4 focus-
visible:outline-[#0077c8] disabled:cursor-not-allowed disabled:opacity-30"
131            >
132              <span aria-hidden="true" className="text-2xl">←</span>
133            </button>
134            <p role="status" aria-atomic="true" className="text-center text-sm tabular-
nums text-gray-600">
```

Annexe — EventsSection.tsx (4)

Source intégrale • explications dans les chapitres 3 à 8

```
135             {startIndex + 1}—{Math.min(startIndex + ITEMS_PER_PAGE, events.length)}
sur {events.length} événements
136         </p>
137         <button
138             type="button"
139             onClick={nextEvents}
140             disabled={startIndex + ITEMS_PER_PAGE >= events.length}
141             aria-label="Événements suivants"
142             aria-controls="events-list"
143             className="flex size-11 shrink-0 items-center justify-center rounded-full
border border-[#0077c8]/30 text-[#0077c8] transition-colors enabled: hover: bg-[#0077c8]
enabled: hover: text-white focus-visible: outline-2 focus-visible: outline-offset-4 focus-
visible: outline-[#0077c8] disabled: cursor-not-allowed disabled: opacity-30"
144         >
145             <span aria-hidden="true" className="text-2xl">→</span>
146         </button>
147     </nav>
148   )}
149 </>
150 )}
151 </div>
152 </section>
153 );
154 }
155
156 type EventCardProps = {
157   event: Event;
158 };
159
160 function EventCard({ event }: EventCardProps) {
161   const [failedImageUrl, setFailedImageUrl] = useState<string | null>(null);
162   const hasImage = Boolean(event.imageUrl && event.imageUrl !== failedImageUrl);
163
164   return (
165     <article className="flex min-w-0 flex-col overflow-hidden rounded-xl border border-
gray-200 bg-white/25 hover: shadow-lg sm: flex-row transition-all duration-300">
166       <div className="flex h-48 shrink-0 items-center justify-center overflow-hidden bg-
sky-50 sm: h-auto sm: min-h-60 sm: w-[38%]">
167         <img
168             src={hasImage ? event.imageUrl! : "/images/logo-couleur.png"}
169             alt=""
170             loading="lazy"
171             onError={hasImage ? () => setFailedImageUrl(event.imageUrl!) : undefined}
172             className={hasImage ? "h-full w-full object-cover" : "h-32 w-32 object-contain
p-3"}
173         />
174       </div>
175
```

Annexe — EventsSection.tsx (5)

Source intégrale • explications dans les chapitres 3 à 8

```
176     <div className="flex min-w-0 flex-1 flex-col justify-center p-5 sm:p-6 md:p-7">
177       <h3 className="mb-3 text-xl font-semibold leading-snug wrap-anywhere text-
gray-800">
178         {event.title}
179       </h3>
180       {event.description && (
181         <p className="mb-5 whitespace-pre-line text-sm leading-relaxed wrap-anywhere
text-gray-600">
182           {event.description}
183         </p>
184       )}
185       <dl className="grid grid-cols-[auto_minmax(0,1fr)] gap-x-3 gap-y-2 text-sm leading-
relaxed">
186         <dt className="font-semibold text-gray-700">Date :</dt>
187         <dd className="wrap-anywhere text-gray-600">
188           <time dateTime={event.startAt}>{event.calendarDateLabel ||
formatDate(event.startAt)}</time>
189         </dd>
190         <dt className="font-semibold text-gray-700">Horaire :</dt>
191         <dd className="text-gray-600">{formatTimeRange(event.startAt, event.endAt)}</dd>
192         <dt className="font-semibold text-gray-700">Lieu :</dt>
193         <dd className="wrap-anywhere text-gray-600">{event.location?.trim() || "À
préciser"}</dd>
194       </dl>
195     </div>
196   </article>
197 );
198 }
199
200 function formatDate(date: string) {
201   return new Intl.DateTimeFormat("fr-FR", {
202     day: "numeric",
203     month: "long",
204     year: "numeric",
205     timeZone: "Europe/Paris",
206   }).format(new Date(date));
207 }
208
209 function formatTime(date: string) {
210   return new Intl.DateTimeFormat("fr-FR", {
211     hour: "2-digit",
212     minute: "2-digit",
213     timeZone: "Europe/Paris",
214   }).format(new Date(date));
215 }
216
217 function formatTimeRange(
218   startAt: string,
```

Annexe — EventsSection.tsx (6)

Source intégrale • explications dans les chapitres 3 à 8

```
219   endAt: string | null
220 ) {
221   if (!endAt) {
222     return formatTime(startAt);
223   }
224
225   return `${formatTime(startAt)} - ${formatTime(endAt)}`;
226 }
```